14

MACHINE LEARNING TEMELLERI

Veri bilimci için kavramsal temel, doğru deney tasarımı ve güvenilir modelleme düşüncesi

Supervised • Unsupervised • Train/Validation/Test • Overfitting • Bias-Variance • Leakage

Veri Bilimci Olma Yolu - Ayrıntılı Konu Anlatımı

0. Bu Modülde Ne Öğreneceksin?

Ana hedef: Bir algoritma ezberlemek değil; bir iş problemini doğru ML problemine çevirebilmek, modeli güvenilir biçimde değerlendirmek ve neden başarısız olabileceğini anlayabilmek.

Bölüm	Kazanım
1-3	ML nedir, hangi problem tiplerinde kullanılır, supervised/unsupervised ayrımı
4-6	Feature-target mantığı, regression/classification, veri bölme stratejileri
7-10	Baseline, preprocessing, loss/metric, model eğitim süreci
11-14	Overfitting, underfitting, bias-variance, regularization
15-17	Data leakage, cross-validation, imbalanced data
18-20	Model seçimi, threshold/calibration, reproducibility
21-23	Uçtan uca vaka, mülakat soruları, çalışma checklisti

Önkoşul: Python/Pandas, temel istatistik ve EDA bilgisi yeterlidir. Bu modülde algoritmaların matematiksel ayrıntısından çok ML çalışma mantığına odaklanıyoruz.

1. Machine Learning Nedir?

Machine Learning (ML), belirli bir görevi yerine getirmek için tüm karar kurallarını elle yazmak yerine, örnek verilerden örüntü öğrenen modeller geliştirme yaklaşımıdır.

Geleneksel programlama	Machine Learning
Kural + veri -> çıktı	Veri + hedef -> öğrenilen model
Kurallar insan tarafından yazılır.	Kurallar/ilişkiler veriden tahmin edilir.
Davranış genellikle deterministiktir.	Çıktı çoğu zaman olasılıksaldır.
Değişen örüntülerde kuralları güncellemek gerekir.	Yeni veri ile yeniden eğitim yapılabilir.

ML kullanmak için temel şart: Geçmiş veride gelecekte de devam etmesi beklenen, ölçülebilir bir sinyal bulunmalıdır. Veri ile hedef arasında öğrenilebilir ilişki yoksa daha karmaşık model çözüm değildir.

2. İş Problemini ML Problemine Çevirmek

İyi bir veri bilimci “hangi modeli kullanalım?” sorusundan önce “hangi kararı iyileştirmeye çalışıyoruz?” sorusunu netleştirir.

1. İş kararını tanımla: Hangi karar daha iyi verilecek?
2. Tahmin anını tanımla: Model hangi anda çalışacak?
3. Target değişkenini tanımla: Ne tahmin ediyoruz?
4. Feature setini tanımla: Tahmin anında gerçekten hangi bilgiler mevcut?
5. Başarı metriğini seç: Teknik metrik hangi iş sonucuyla ilişkili?
6. Baseline belirle: Model neyi geçmek zorunda?
7. Kısıtları belirle: latency, maliyet, açıklanabilirlik, fairness, veri erişimi.

İş sorusu	ML formülasyonu	Problem tipi
Müşteri önümüzdeki 30 günde churn eder mi?	Her müşteri için churn olasılığı	Binary classification
Gelecek ay satış kaç adet olacak?	SKU-mağaza bazında miktar tahmini	Regression / forecasting
Benzer müşterileri bulabilir miyiz?	Davranış vektörlerine göre gruplama	Clustering
Hangi ürünü önce göstermeliyiz?	Kullanıcı-ürün skorları / sıralama	Recommendation / ranking

Kritik soru: Modelin tahmin yaptığı anda hedefe ait gelecekteki bilgileri kullanıyor musun? Eğer evetse leakage riski vardır.

3. Öğrenme Türleri

Tür	Etiket var mı?	Amaç	Örnek
Supervised learning	Evet	X -> y eşlemesini öğrenmek	Churn, fiyat, fraud
Unsupervised learning	Hayır	Yapı/segment/temsil keşfetmek	Clustering, PCA
Semi-supervised	Kısmen	Az etiket + çok etiketsiz veriden yararlanmak	Görüntü/metin sınıflama
Self-supervised	Harici etiket yok	Verinin kendisinden öğrenme görevi üretmek	Modern representation learning

- Supervised öğrenmede değerlendirme için doğru etiketlere ihtiyaç vardır.
- Unsupervised çıktılar “doğru/yanlış” yerine yararlılık, stabilite ve iş anlamı ile değerlendirilir.
- ML projesinin tipi yalnızca algoritmayı değil, veri toplama ve değerlendirme tasarımını da belirler.

4. Feature, Target ve Observation

Kavram	Anlam	Örnek
Observation / row	Modelin bir karar/tahmin birimi	Bir müşteri, işlem, gün
Feature (X)	Tahmin anında kullanılabilen girdi	Son 30 gün sipariş sayısı
Target (y)	Modelin öğrenmeye çalıştığı çıktı	30 gün içinde churn=1
Prediction	Modelin ürettiği tahmin	P(churn)=0.73

Point-in-time correctness: Bir feature yalnızca tahmin anında bilinen veriden hesaplanmalıdır. Örneğin 1 Ocak tarihinde churn tahmini yapıyorsan 5 Ocak sipariş bilgisini kullanamazsın.

```
# Kavramsal ayrım
X = df[["orders_30d", "days_since_last_order", "avg_basket"]]
y = df["churn_30d"]

# Model X'ten y'yi öğrenir.
model.fit(X_train, y_train)
p = model.predict_proba(X_test)[:, 1]
```

5. Regression ve Classification

Özellik	Regression	Classification
Target	Sürekli sayısal	Kategori / sınıf
Örnek	Fiyat, talep, süre	Churn, fraud, spam
Çıktı	Sayısal tahmin	Sınıf veya sınıf olasılığı
Tipik metrik	MAE, RMSE, R^2	Precision, Recall, F1, ROC-AUC, PR-AUC
Sık hata	Outlier etkisini göz ardı etmek	Accuracy'ye aşırı güvenmek

Önemli: Classification modeli çoğu zaman önce olasılık üretir. "Fraud / değil" gibi son karar, seçilen threshold ile oluşur. Bu nedenle model ile karar kuralını ayrı düşün.

6. Train - Validation - Test

Aynı veri üzerinde modeli hem eğitip hem de başarısını ölçersen, modelin yeni veri performansını güvenilir şekilde tahmin edemezsin. Veri bölme işlemi genelleme performansını ölçmek içindir.

Set	Ne için kullanılır?	Modele etkisi
Train	Parametreleri öğrenmek	Doğrudan eğitim
Validation	Model/feature/hyperparameter seçmek	Dolaylı olarak model seçimini etkiler
Test	En sonda, tarafsız final değerlendirme	Model geliştirme kararlarında kullanılmamalı

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)
```

Altın kural: Test setini geliştirme boyunca tekrar tekrar kontrol etmek, test setini validation setine dönüştürür. Gerçek final performans tahmini bozulur.

7. Random Split Her Zaman Doğru Değildir

Veri yapısı	Yanlış yaklaşım	Doğru yaklaşım
Zaman serisi	Gelecek kayıtları train'e karıştırmak	Geçmiş -> train, gelecek -> validation/test
Aynı kullanıcının çok satırı	Aynı kullanıcı train ve testte	Group split
Nadir sınıf	Sınıf oranlarını bozmak	Stratified split
Coğrafi genelleme	Aynı bölgeyi karıştırmak	Bölge bazlı holdout gerekebilir

Amaç: Split tekniği üretimde karşılaşacağın "yeni veri" koşulunu taklit etmelidir.

8. Baseline: Modelin Geçmesi Gereken Çıta

Baseline, karmaşık modelin gerçekten değer üretip üretmediğini anlamanı sağlar. Baseline olmadan “0.82 skor aldım” tek başına anlamlı değildir.

Problem	Basit baseline
Regression	Ortalama / medyan tahmini
Binary classification	Çoğunluk sınıfı veya sabit olasılık
Time series	Naive: son değer / geçen hafta aynı gün
Recommendation	En popüler ürünler

```
from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(strategy="prior")
baseline.fit(X_train, y_train)
print(baseline.score(X_val, y_val))
```

Pratik yorum: Model baseline’i çok az geçiyorsa önce daha karmaşık algoritma değil; veri kalitesi, target tanımı, feature’lar ve problem formülasyonu sorgulanmalıdır.

9. Preprocessing Neden Pipeline İçinde Olmalı?

- Eksik değer doldurma (imputation)
- Kategorik değişken encoding
- Ölçekleme / standardization
- Feature engineering
- Feature selection

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

# Fit işlemleri yalnızca train verisinde öğrenilmeli.
# Pipeline bu sınırı korumayı kolaylaştırır.
```

Leakage örneği: Tüm dataset üzerinde StandardScaler.fit() yapıp sonra train/test ayırmak, test dağılımına ait bilgiyi train sürecine sızdırır.

10. Loss Function ve Evaluation Metric

Modelin eğitim sırasında optimize ettiği fonksiyon ile iş başarısını raporladığın metrik aynı olmak zorunda değildir.

Kavram	Rol	Örnek
Loss	Model parametrelerinin öğrenilmesini yönlendirir	Log loss, MSE
Metric	Modeli değerlendirmek / karşılaştırmak	F1, ROC-AUC, MAE
Business KPI	Gerçek iş etkisini ölçmek	Kazanç, maliyet, churn azalması

Örnek: Fraud modelinde recall yükselirken false positive sayısı da artabilir. Teknik metrik iyileşse bile manuel inceleme maliyeti aşırı yükseliyorsa iş sonucu kötüleşebilir.

11. Overfitting Nedir?

Overfitting, modelin eğitim verisinin gerçek sinyalini öğrenmenin yanında gürültü ve tesadüfi ayrıntıları da ezberlemesi; bu nedenle yeni veride performans kaybetmesidir.

Belirti	Yorum
Train çok iyi, validation belirgin kötü	Klasik overfitting sinyali
Model çok karmaşık	Variance artabilir
Az veri + çok feature	Ezberleme riski büyür
CV skorları çok değişken	Model/veri instabilitesi olabilir

- Daha fazla ve daha temsil edici veri
- Daha basit model
- Regularization
- Feature selection
- Cross-validation
- Early stopping (uygun modellerde)

12. Underfitting Nedir?

Underfitting, modelin verideki temel yapıyı dahi yakalayamayacak kadar basit veya yetersiz olmasıdır. Train ve validation performansı birlikte kötüdür.

Durum	Train	Validation	Muhtemel teşhis
A	Kötü	Kötü	Underfitting / zayıf sinyal
B	Çok iyi	Kötü	Overfitting
C	İyi	İyi	Dengeli genelleme

13. Bias - Variance Trade-off

Bias ve variance, model hatasını düşünmek için kullanılan iki temel kavramdır. Basitleştirilmiş biçimde: yüksek bias modelin sistematik olarak yetersiz kalması; yüksek variance ise veri örneği değiştiğinde model davranışının fazla değişmesidir.

Model davranışı	Bias	Variance	Tipik sonuç
Aşırı basit	Yüksek	Düşük	Underfitting
Dengeli	Orta/düşük	Kontrollü	İyi genelleme
Aşırı karmaşık	Düşük	Yüksek	Overfitting

Düşünce modeli: “Daha karmaşık model daha iyidir” doğru değildir. Amaç train hatasını minimuma indirmek değil, görülmemiş veride beklenen hatayı azaltmaktır.

14. Regularization

Regularization, modelin aşırı karmaşık çözümlere yönelmesini cezalandırarak genelleme performansını iyileştirmeye çalışan yöntemler ailesidir.

Yöntem	Temel fikir
L2 / Ridge	Büyük katsayıları yumuşak biçimde cezalandırır
L1 / Lasso	Bazı katsayıları sıfıra çekebilir
Tree constraints	max_depth, min_samples_leaf gibi sınırlar
Early stopping	Validation kötüleşmeye başladığında eğitimi durdurur
Dropout (DL)	Eğitim sırasında rastgele bağlantıları geçici kapatır

Denge: Regularization çok zayıfsa overfitting devam eder; çok güçlüyse model underfit olabilir.

15. Data Leakage: En Tehlikeli Hatalardan Biri

Leakage, modelin gerçek tahmin anında erişemeyeceği bilgiyi eğitim veya değerlendirme sırasında kullanmasıdır. Sonuç: offline skorlar olağanüstü görünür, production performansı çöker.

Leakage türü	Örnek
Target leakage	Churn tahmininde "hesap kapatma tarihi" feature'ı
Temporal leakage	Gelecek haftanın bilgisini geçmiş tahmininde kullanmak
Preprocessing leakage	Scaler/imputer'ı tüm veride fit etmek
Duplicate/entity leakage	Aynı müşteri train ve testte
Aggregation leakage	Feature hesaplamasına hedef sonrası olayları dahil etmek

Leakage testi: Her feature için şu soruyu sor: "Bu tahmini gerçek hayatta tam bu anda yaparken bu bilgi elimde olur muydu?"

16. Cross-Validation

Tek bir train-validation ayrımı tesadüfi olarak kolay veya zor olabilir. Cross-validation, birden fazla split üzerinden performans dağılımını görmeyi sağlar.

Yöntem	Ne zaman?
KFold	Bağımsız ve dengeli örnekler
StratifiedKFold	Classification ve sınıf oranını koruma ihtiyacı
GroupKFold	Aynı entity'nin fold'lar arasında ayrılması gerektiğinde
TimeSeriesSplit	Zamansal sıranın korunması gerektiğinde

```
from sklearn.model_selection import cross_validate, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
res = cross_validate(model, X, y, cv=cv, scoring=["roc_auc", "f1"])
print(res["test_roc_auc"].mean(), res["test_roc_auc"].std())
```

17. Imbalanced Data

Fraud, churn veya arıza gibi problemlerde pozitif sınıf az olabilir. Bu durumda accuracy yanıltıcı hale gelebilir.

Gerçek durum	Accuracy	Neden sorunlu?
%1 fraud, model herkese “normal” diyor	%99	Fraud yakalama değeri sıfır

- Confusion matrix ile FP/FN maliyetlerini incele.
- Precision, recall, F1 ve özellikle nadir pozitif sınıfta PR-AUC düşün.
- Class weight, resampling veya threshold tuning seçeneklerini problem maliyetine göre değerlendir.
- Resampling yapıyorsan yalnızca training fold içinde uygula; test dağılımını değiştirme.

Asıl soru: Hangi hata daha pahalı: false positive mi false negative mi? Metrik ve threshold seçimi bu maliyete dayanmalıdır.

18. Model Seçimi: “En Yüksek Skor”dan Fazlası

Boyut	Sorulacak soru
Predictive performance	Validation/CV performansı yeterli mi?
Stability	Farklı fold/segment/zamanlarda tutarlı mı?
Latency	Gerekli sürede tahmin yapabiliyor mu?
Explainability	Kararların açıklanması gerekiyor mu?
Cost	Training/inference maliyeti kabul edilebilir mi?
Maintenance	Sistem sürdürülebilir mi?

Production bakışı: 0.002 AUC kazancı için inference maliyetini 10 kat artıran model her zaman daha iyi model değildir.

19. Probability, Threshold ve Calibration

Classification modelinin 0.8 olasılık üretmesi ile 1 sınıfına karar vermesi farklı şeylerdir. Threshold, iş maliyetine göre seçilen bir karar kuralıdır.

Threshold düşerse	Threshold yükselirse
Daha fazla pozitif tahmin	Daha az pozitif tahmin
Recall genellikle artar	Precision genellikle artabilir
False positive artabilir	False negative artabilir

Calibration: Modelin 0.80 dediği örneklerin yaklaşık %80'i gerçekten pozitif oluyorsa olasılıklar iyi kalibre edilmiş kabul edilir. Risk skoru kullanılan sistemlerde calibration kritik olabilir.

20. Reproducibility ve Deney Disiplini

- Random seed kullan ama sadece seed'e güvenme.
- Dataset versiyonunu ve split mantığını kaydet.
- Feature listesini ve preprocessing adımlarını version-control altında tut.
- Model parametrelerini, metrikleri ve deney sonuçlarını kaydet.
- Test setini son değerlendirme dışında kullanma.
- Aynı kodun aynı veri üzerinde yeniden çalışabilmesini hedefle.

```
experiment = {
    "data_version": "2026-10-01",
    "split": "time_based_80_10_10",
    "model": "logistic_regression",
    "C": 0.5,
    "seed": 42,
    "val_roc_auc": 0.812
}
```

21. Uçtan Uca Vaka: Churn Prediction

Senaryo: Bir abonelik şirketi, hangi müşterilerin önümüzdeki 30 gün içinde aboneliğini iptal edeceğini tahmin etmek istiyor. Amaç retention ekibinin sınırlı teklif bütçesini yüksek riskli müşterilere yönlendirmek.

Adım	Tasarım
Observation	Her müşteri için haftalık snapshot
Prediction time	Her pazartesi 08:00
Target	Sonraki 30 gün içinde iptal=1
Features	Son 90 gün kullanım, ödeme, destek, engagement
Split	Zamana dayalı
Baseline	Geçmiş churn oranı / basit logistic regression
Primary metric	PR-AUC + recall@budget
Guardrail	Yanlış pozitiflere verilen teklif maliyeti

21.1. Doğru geliştirme akışı

1. EDA ve target tanımını doğrula.
2. Point-in-time feature üretimini doğrula.
3. Time-based train/validation/test oluştur.
4. Basit baseline kur.
5. Preprocessing + model pipeline oluştur.
6. Validation üzerinde model/feature/hyperparameter kararlarını ver.
7. Threshold'u retention bütçesi ve hata maliyetiyle seç.
8. Sadece finalde test setini aç.
9. Segment ve zaman stabilitesini incele.
10. Production monitoring için feature/score/target metriklerini tanımla.

21.2. Bu vakadaki olası leakage

Feature	Durum
Son 30 gün kullanım sayısı	Tahmin zamanından önce hesaplanıyorsa uygun
İptal talebi açıldı mı?	Target'a çok yakın / leakage olabilir
İptal tarihi	Kesin target leakage
30 gün sonraki ödeme sonucu	Temporal leakage

22. Sık Yapılan Hatalar

Hata	Neden kötü?	Doğru yaklaşım
Model seçerek başlamak	Problem/target yanlış olabilir	Önce karar ve target
Tüm veriyi preprocess etmek	Leakage	Pipeline + train-only fit
Test skoruna göre tuning	Test tarafsızlığını kaybeder	Validation/CV
Accuracy'ye güvenmek	Imbalance'da yanıltıcı	Maliyet + uygun metrik
Random split kullanmak	Zaman/entity leakage olabilir	Üretimi taklit eden split
Baseline kurmamak	Skorun değeri bilinmez	Basit referans model
Tek metriğe bakmak	İş risklerini gizleyebilir	Metric set + business KPI
En karmaşık modeli seçmek	Maliyet/maintenance artar	Value vs complexity

23. Mülakat Soruları ve Kısa Cevap Çerçeveleri

Soru	Cevapta geçmesi gerekenler
Overfitting nedir?	Train-validation gap, noise ezberleme, genelleme, regularization/CV
Train/validation/test neden ayrı?	Fit, model selection, tarafsız final estimate
Data leakage nedir?	Tahmin anında olmayan bilginin modele sızması
Accuracy ne zaman kötü metriktir?	Imbalance ve asimetrik hata maliyetlerinde
Cross-validation neden kullanılır?	Tek split belirsizliğini azaltmak, stabilite görmek
Random split ne zaman yanlış?	Time series, repeated entities, grouped data
Baseline neden gerekir?	ML modelinin gerçek incremental value'sunu ölçmek
Bias-variance trade-off nedir?	Yetersiz model vs aşırı hassas model dengesi

Mülakat yaklaşımı: Tanım -> neden önemli -> küçük örnek -> çözüm/önlem sırasıyla cevap verirsen teknik bilgin daha net görünür.

24. Mini Alıştırmalar

1. %0.5 fraud oranı olan bir veride model %99.5 accuracy alıyor. Bu sonucu nasıl değerlendirirsin?
2. Aynı müşteri farklı tarihlerde 20 satırla bulunuyor. Train-test split'i nasıl yaparsın?
3. Churn modelinde "account_closed_date" feature'ı AUC'yi 0.72'den 0.99'a çıkarıyor. Ne düşünürsün?
4. Train F1=0.96, validation F1=0.61. Üç olası neden ve çözüm yaz.
5. Bir model ROC-AUC=0.90, diğer model 0.895. İlk model inference'da 12 kat pahalı. Hangisini seçersin? Hangi bilgiler eksik?
6. Validation performansı 5 fold'da [0.82, 0.81, 0.83, 0.65, 0.84]. Ne araştırırsın?
7. Fraud tespitinde false negative maliyeti false positive'den 100 kat fazla. Threshold seçimine etkisini açıkla.

25. Final Checklist

Kontrol	Evet/Hayır
İş kararı ve prediction time açık mı?	☐
Target kesin ve ölçülebilir mi?	☐
Feature'lar prediction time'da mevcut mu?	☐
Split üretim senaryosunu taklit ediyor mu?	☐
Baseline var mı?	☐
Preprocessing yalnızca training verisinde fit ediliyor mu?	☐
Metrik iş maliyetiyle uyumlu mu?	☐
Overfitting/underfitting kontrol edildi mi?	☐
Cross-validation uygun şekilde kullanıldı mı?	☐
Leakage kontrolleri yapıldı mı?	☐
Threshold karar maliyetine göre seçildi mi?	☐
Test seti yalnızca finalde kullanıldı mı?	☐
Deney tekrar üretilebilir mi?	☐

Bu modülün özeti: Machine Learning'in özü algoritma listesi değil; doğru problem tanımı, doğru veri sınırı, güvenilir değerlendirme ve genelleme düşüncesidir. Bu temel oturduğunda sonraki regression, classification ve ensemble modülleri çok daha anlamlı hale gelir.

26. Sonraki Modüllere Geçiş Haritası

Bu modülde öğrendiğin temel	Bir sonraki kullanım
Train/validation/test + CV	Regression ve classification model seçimi
Loss vs metric	Model evaluation
Overfitting + regularization	Linear models, trees, boosting
Leakage + pipeline	Feature engineering ve ML pipelines
Imbalance + threshold	Fraud/churn gibi classification vakaları
Baseline + reproducibility	Hyperparameter optimization ve production ML

Önerilen pratik: Bu PDF'den sonra küçük bir churn veya loan-default datasetinde önce yalnızca baseline + split + metric + leakage kontrolü kur. Algoritma tuning'e geçmeden ML düşünce sistemini pratik et.